

课程笔记

CS224N Lecture 9: Pretraining

整理：AI Course Notes Contributors；授课：Chris Manning

2024 年

课程版本： 2024 年中文讲义整理版

笔记来源： AI Course Notes Contributors

本版处理： 移除课件截图和对应图注，不包含 Stanford 原始课件、图片或视频。

授权： 笔记依 CC BY-NC-SA 4.0 分享；完整许可与改动记录见下一页。

来源与许可

本中文笔记由 AI Course Notes Contributors 整理，课程版本为 2024；源稿：<https://github.com/hqhq1025/ai-course-notes/tree/717e2df65c3d2eee593c171156a30d4f8f6812b9/cs224n/lecture09>。

笔记内容依 CC BY-NC-SA 4.0 分享：<https://creativecommons.org/licenses/by-nc-sa/4.0/>。

本副本的改动：移除 Stanford 原始课件截图与依赖这些截图的图注，移除课程视频封面/链接，并清理 TeX 源稿中的行尾空白后重新编译为可独立阅读的 PDF；同时加入本来源与许可说明。完整许可文本随本文件夹提供。Stanford 课件、课件图片和视频不包含在该授权内；请从对应年份 Stanford 课程页访问原始课件。

目录

1 子词分词：从有限词表到开放词表	3
1.1 固定词表的问题	3
1.2 Byte Pair Encoding (BPE) 算法	3
1.3 本章小结	4
2 从词嵌入到全模型预训练	4
2.1 分布式假设的两个层次	4
2.2 旧范式 vs 新范式	4
2.3 预训练的核心思想：重构输入	5
2.4 预训练-微调范式	5
2.5 本章小结	6
3 Encoder 预训练：BERT 与掩码语言建模	6
3.1 三种 Transformer 架构回顾	6
3.2 为什么 Encoder 不能用语言建模预训练?	6
3.3 BERT：掩码语言建模	6
3.4 下一句预测任务 (NSP)	7
3.5 BERT 在 GLUE 基准上的表现	7
3.6 BERT 的微调方式	7
3.7 BERT 的局限性	8
3.8 BERT 的改进：SpanBERT 与 RoBERTa	8
3.9 参数高效微调	8
3.10 本章小结	9
4 Encoder-Decoder 预训练：T5 与片段腐蚀	9
4.1 Encoder-Decoder 的预训练目标	9
4.2 T5 与片段腐蚀	9
4.3 T5 的“隐式知识检索”能力	9
4.4 本章小结	10
5 Decoder 预训练：GPT 系列与大语言模型	10
5.1 Decoder 的预训练与微调	10
5.2 GPT-1：开创性的预训练 Decoder	10
5.3 GPT-2：惊人的生成能力	10
5.4 GPT-3：涌现的上下文学习能力	11
5.5 Scaling Laws 与计算效率	11
5.6 Chain-of-Thought 提示	11
5.7 为什么 Decoder-only 模型占据主导地位?	12
5.8 本章小结	12

6	预训练数据与伦理考量	12
6.1	预训练数据的来源	12
6.2	预训练学到了什么——以及它的阴暗面	13
6.3	本章小结	13
7	总结与延伸	13
7.1	核心知识图谱	13
7.2	关键 Takeaways	14
7.3	拓展阅读	14

1 子词分词：从有限词表到开放词表

在课程的前几讲中，我们假设模型拥有一个**固定的词表** V 。每个单词被映射到一个唯一的向量表示。然而，这种设计面临一个根本性问题：语言是**开放的**，新词、拼写变体、形态变化不断出现，固定词表无法覆盖所有可能的输入。

1.1 固定词表的问题

当模型遇到训练时未见过的字符串时，只能将其映射为一个通用的 `<UNK>` (unknown) 标记。这意味着：

- 所有未知词共享同一个向量，**完全丢失了语义信息**
- 在形态丰富的语言（如斯瓦希里语、芬兰语、土耳其语）中，一个动词可能有数百种变形，词表膨胀极为严重
- 即使是英语，拼写变体、新造词、专有名词也层出不穷

1.2 Byte Pair Encoding (BPE) 算法

现代 NLP 的解决方案是**子词分词** (subword tokenization)。其核心思想是：不以“词”为最小单位，而是以**频繁共现的字符子串**为基本单元。

BPE 算法流程

1. 以所有单个字符（加上词尾标记）初始化词表
2. 统计训练语料中所有相邻字符对的共现频率
3. 将频率最高的字符对合并为一个新符号，加入词表
4. 用新符号替换语料中的对应字符对
5. 重复步骤 2-4，直到词表达达到预设大小（通常 30k-50k）

分词时，算法尽量用**最少的子词**来表示输入。例如：

- 常见词“hat”直接作为一个 token
- 不常见的“taaaasty”被拆分为“ta” + “##aa” + “##sty”三个子词 token
- “transformerify”可能被拆分为“transformer” + “##ify”

其中“##”前缀表示这个子词**不以空格开头**，即它是前一个子词的延续。

⁰Slides 第 6 页。来源：Wiktionary。

子词分词的关键性质

- 系统**不区分**一个 token 是完整的词还是子词片段——它们在模型中的地位完全相同，都是嵌入矩阵中的一个索引
- 如果需要获取一个被拆分的词的表示，可以**取平均**或取最后一个子词的上下文表示
- 现代 LLM（GPT、BERT、T5 等）几乎全部使用 BPE 或类似的子词分词方案
- 序列长度是 Transformer 的瓶颈，因此分词时倾向于用**更少更大的子词**

子词分词不是“聪明”的语素切分

BPE 是纯粹基于统计频率的算法，**不理解语言学意义上的词素** (morpheme)。它可能把 “un-happiness” 切分为 “un” + “happiness” (恰好对应语素)，也可能切分为 “unh” + “appiness” (无语言学意义)。切分结果完全取决于训练语料的统计分布。

1.3 本章小结

子词分词解决了固定词表的 UNK 问题，使模型能够处理任意输入字符串。BPE 通过迭代合并频繁共现字符对来构建词表，在词表大小和序列长度之间取得平衡。这一技术已成为所有现代语言模型的标配。

2 从词嵌入到全模型预训练

2.1 分布式假设的两个层次

课程开篇曾引用 J.R. Firth 的名言：“You shall know a word by the company it keeps.” 这是**分布式假设**的经典表述，也是 Word2Vec 的理论基础。

但同一位语言学家还有另一句话：“The complete meaning of a word is always contextual, and no study of meaning apart from a complete context can be taken seriously.” 这揭示了一个更深层的洞察：词义不仅由其典型的共现伙伴决定，更由其**具体出现的上下文**决定。

Word2Vec 的根本局限

Word2Vec 为每个词学习一个**固定的向量**，无论它出现在什么上下文中。对于多义词（如英语的 “record”：动词 “记录” / 名词 “唱片”），这个向量只能是所有义项的**混合体**，无法在特定语境中区分不同含义。

2.2 旧范式 vs 新范式

在预训练出现之前，NLP 的典型工作流程是：

1. 用 Word2Vec / GloVe 训练词嵌入（预训练的**仅仅是嵌入层**）
2. 在嵌入层之上堆叠一个随机初始化的 LSTM 或 Transformer
3. 用下游任务（如情感分析、机器翻译）的标注数据从头训练上层网络

新范式的革命性变化在于：**预训练整个网络**（嵌入层 + Transformer），而不仅仅是词嵌入。这带来了三大优势：

1. **上下文表示**：“record”在不同语境中获得不同的向量
2. **强参数初始化**：所有参数都从一个“好的起点”开始，而非随机初始化
3. **语言概率分布**：模型学到了自然语言的生成概率，可以直接用于文本生成

2.3 预训练的核心思想：重构输入

预训练的核心假设是：如果我们遮住文本的一部分，让神经网络来**重构被遮住的内容**，那么网络为了做好这个任务，必须学会大量关于语言和世界的知识。

通过这种“填空”任务，模型可以隐式地学到：

- **世界知识** (trivia): “Stanford University is located in Stanford/Palo Alto, California”
- **句法** (syntax): “I put the/a fork down on the table”
- **共指** (coreference): “The woman walked across the street, checking for traffic over her shoulder”
- **语义** (semantics): “I went to the ocean to see the fish, turtles, seals, and whales”
- **情感** (sentiment): “The movie was bad/great”
- **推理** (reasoning): “Iroh went into the kitchen... Zuko left the kitchen”

2.4 预训练-微调范式

预训练-微调范式 (Pretrain-Finetune Paradigm)

1. **预训练**：在海量无标注文本上训练，学习语言的通用表示。目标是**最小化预训练损失**
$$\hat{\theta} = \arg \min_{\theta} \mathcal{L}_{\text{pretrain}}(\theta)$$
2. **微调**：以 $\hat{\theta}$ 为起点，在下游任务的标注数据上继续训练。梯度下降从 $\hat{\theta}$ 出发，走向对当前任务最优的参数 θ^*

为什么预训练的参数初始化如此重要？Chris Manning 强调：“If you could actually solve this min, the starting point shouldn't matter. But it really, really, really does.” 直觉上：

- 梯度下降在微调过程中会停留在 $\hat{\theta}$ 附近的局部最优
- $\hat{\theta}$ 附近的参数空间已经“学会了语言”，因此该区域的局部最优往往**泛化能力更强**
- 无标注数据的量级（万亿词级别）远超标注数据（百万词级别），预训练利用了这个巨大的信息优势

预训练的数据优势不仅在于量

即使你拥有海量的情感分析标注数据，从头训练的模型也可能不如预训练 + 微调的模型。原因在于：

- 单一任务的标注数据多样性有限，模型容易过拟合到特定分布
- 预训练数据涵盖了互联网上极其多样的文本类型，使模型具备更强的**分布外泛化**（out-of-distribution generalization）能力
- 语言建模本身是一个极其困难的任务，要求模型学习语言的方方面面

2.5 本章小结

从 Word2Vec 的“仅预训练嵌入”到“预训练整个网络”，NLP 经历了一次范式转变。预训练-微调范式的成功基于两个关键因素：（1）海量无标注文本提供了丰富的自监督信号；（2）预训练学到的参数提供了一个泛化能力极强的初始化点。这一范式至今仍是 NLP 的主流方法论。

3 Encoder 预训练：BERT 与掩码语言建模

3.1 三种 Transformer 架构回顾

预训练方法与 Transformer 的架构类型密切相关。Chris Manning 将模型分为三类：

三种架构的特点与适用场景

- **Encoder**：每个 token 能看到整个序列（双向上下文）。适合**理解类任务**（分类、NER、问答抽取）。代表模型：BERT
- **Encoder-Decoder**：Encoder 部分看到完整输入，Decoder 部分自回归生成输出。适合**序列到序列任务**（翻译、摘要）。代表模型：T5
- **Decoder**：每个 token 只能看到它之前的 token（单向上下文）。适合**生成类任务**。代表模型：GPT 系列

3.2 为什么 Encoder 不能用语言建模预训练？

Encoder 拥有双向上下文，这意味着在预测下一个词时，模型已经**看到了**那个词——预测任务变得毫无意义（trivial）。

解决方案：**掩码语言建模**（Masked Language Modeling, MLM）——先遮住一些词，再让模型预测被遮住的词。

3.3 BERT：掩码语言建模

BERT（Bidirectional Encoder Representations from Transformers, 2018）是第一个成功将掩码语言建模大规模应用于预训练的模型。

BERT 的掩码策略选取 15% 的 token 进行处理，对于被选中的 token：

- 80% 替换为 [MASK] 标记

- 10% 替换为词表中的随机词
- 10% 保持不变

为什么不能全部用 [MASK] ?

如果只在 [MASK] 标记处做预测，模型可能会学到：“只有遇到 [MASK] 时才需要认真构建表示，其他位置可以偷懒”。但在实际使用时（如情感分析），输入中**不会出现** [MASK] 标记。通过混合使用随机替换和保持不变，模型被迫在**任何位置**都构建高质量的上下文表示。

3.4 下一句预测任务 (NSP)

BERT 原始论文还引入了一个辅助任务——**下一句预测** (Next Sentence Prediction, NSP)：给定两段文本 A 和 B，预测 B 是否是 A 的真实续文。

这个任务的设计初衷是教会模型理解**长距离的文本连贯性**。然而，后续研究 (RoBERTa, 2019) 发现这个任务**并不必要**，原因有二：

1. NSP 将有效上下文长度减半（原本 512 token 的窗口被分成两段各 256 token）
2. 模型在 NSP 任务上表现很差，可能是因为这个任务对当时的模型来说太难了

3.5 BERT 在 GLUE 基准上的表现

BERT 的发布在 NLP 领域引起了巨大震动。在 GLUE 基准（包含多种 NLP 任务）上，BERT 全面超越了之前**为每个任务精心设计的专用模型**。

Chris Manning 评价道：“All of that was blown out of the water by just build a big Transformer and teach it to predict the missing words.” BERT 的成功宣告了“为每个任务设计专用架构”时代的终结。

BERT 的核心规格

- **BERT_{BASE}**：110M 参数，12 层，768 维隐层，12 头
- **BERT_{LARGE}**：340M 参数，24 层，1024 维隐层，16 头
- **训练数据**：BookCorpus + English Wikipedia（约 25 亿词）
- **预训练计算**：在当时被认为“只有 Google 才负担得起”，但今天看来并不算大
- **微调**：在单个 GPU 上即可完成

3.6 BERT 的微调方式

将预训练好的 BERT 用于下游任务非常简单：

1. 移除预训练时的 MLM 预测头
2. 对于分类任务：取 [CLS] 标记或最后一个 token 的输出向量，接一个线性分类器

⁰Slides 第 35 页。来自 Devlin et al., 2018。

⁰Slides 第 36 页。来自 Devlin et al., 2018。

3. 对于 token 级别任务（如 NER）：对每个 token 的输出向量接线性分类器
4. 整个网络（BERT + 新分类器）一起微调

3.7 BERT 的局限性

BERT 不适合文本生成

BERT 的双向上下文使其非常擅长“理解”任务（分类、抽取式问答、NER），但它**没有自回归生成的能力**——无法像语言模型那样逐词生成文本。如果你需要生成摘要、翻译或对话回复，应该使用 Encoder-Decoder 或 Decoder-only 模型。

3.8 BERT 的改进：SpanBERT 与 RoBERTa

SpanBERT：用**连续片段掩码**（span masking）替代随机单词掩码。随机掩码单个子词过于简单（前后子词的线索太强），而掩码一整个连续片段迫使模型进行更深层的理解。

RoBERTa（Robustly Optimized BERT Approach）：

- 移除了 NSP 任务
- 在更多数据上训练更长时间
- 动态掩码（每次 epoch 重新生成掩码位置）
- 是 BERT 的**直接替代品**，性能更优

选择建议

如果你的项目需要使用 encoder 模型（如分类、NER、抽取式问答），优先使用 **RoBERTa** 而非原始 BERT，它在几乎所有任务上都表现更好，且完全兼容 BERT 的使用方式。

3.9 参数高效微调

当模型变得越来越大时，全参数微调变得昂贵且不实际。**参数高效微调**（Parameter-Efficient Fine-Tuning, PEFT）只调整少量参数，而冻结大部分预训练参数。

三种主要的参数高效微调方法：

1. **Prefix Tuning / Prompt Tuning**：冻结网络，在序列前添加可训练的“伪词向量”
2. **Adapter Layers**：在 Transformer 层之间插入小型可训练的适配层
3. **LoRA**（Low-Rank Adaptation）：冻结原始权重矩阵 W ，学习一个低秩增量 $\Delta W = BA$ ，令 $W' = W + BA$

LoRA 的核心思想

对于权重矩阵 $W \in \mathbb{R}^{d \times d}$ ，LoRA 学习两个小矩阵 $B \in \mathbb{R}^{d \times r}$ 和 $A \in \mathbb{R}^{r \times d}$ （ $r \ll d$ ），令实际权重为：

$$W' = W + BA$$

只训练 B 和 A （参数量从 d^2 降至 $2dr$ ），冻结 W 。这背后的直觉是：从预训练到下游任务的参数变化是**低秩的**，不需要修改全部参数。

3.10 本章小结

Encoder 预训练以 BERT 为代表，通过掩码语言建模让模型学会双向上下文表示。BERT 在各类 NLP 基准上的巨大成功标志着“预训练-微调”范式的确立。后续的 SpanBERT、RoBERTa 在训练策略上进行了优化。参数高效微调方法（Prefix Tuning、LoRA）则解决了大模型微调的效率问题。但 Encoder 模型不适合生成任务，这引出了 Encoder-Decoder 和 Decoder-only 架构。

4 Encoder-Decoder 预训练：T5 与片段腐蚀

4.1 Encoder-Decoder 的预训练目标

Encoder-Decoder 架构可以简单地用**语言建模**来预训练：将文本前半部分交给 Encoder（提供双向上下文），用 Decoder 自回归生成后半部分。

但在实践中，一种更有效的方法是**片段腐蚀**（Span Corruption）。

4.2 T5 与片段腐蚀

T5（Text-to-Text Transfer Transformer）将所有 NLP 任务统一建模为“文本到文本”的格式，并使用片段腐蚀作为预训练目标。

片段腐蚀的工作方式：

1. 在输入中随机遮住若干**连续片段**，用特殊标记（如 [X]、[Y]）替代
2. Encoder 处理带遮罩的输入（保持双向上下文）
3. Decoder 自回归**生成**被遮住的内容：先输出 [X]，再输出对应片段，再输出 [Y]，再输出对应片段...

片段腐蚀

片段腐蚀结合了两个世界的优点：

- 像 BERT 一样利用**双向上下文**来理解输入
- 像语言模型一样以**自回归方式生成**输出

这使得预训练好的 T5 可以自然地应用于翻译、摘要等序列到序列任务。

4.3 T5 的“隐式知识检索”能力

一个令人惊讶的发现是：T5 在预训练中看过大量类似“Franklin D. Roosevelt was born in [MASK]”的文本，之后经过在问答数据上微调，它能够**直接从参数中**回忆出答案，而非从外部知识库检索。

这种“闭卷”问答能力表明，预训练模型在其参数中**隐式存储了大量事实知识**。

⁰Slides 第 41 页。来自 Raffel et al., 2018。

模型回答总是看起来很流畅，但经常是错的

Chris Manning 特别指出：这些模型的回答 “always look very fluent, always look very reasonable, but they’re frequently wrong.” 这个问题（后来被称为“幻觉”，hallucination）至今仍是大型语言模型的核心挑战之一。

4.4 本章小结

Encoder-Decoder 预训练以 T5 为代表，通过片段腐蚀实现了双向理解与生成能力的统一。T5 将所有任务转化为“文本到文本”格式，提供了一种优雅的统一框架。模型在预训练过程中隐式存储了大量事实知识，但也伴随着幻觉问题。

5 Decoder 预训练：GPT 系列与大语言模型

5.1 Decoder 的预训练与微调

Decoder-only 模型的预训练目标就是标准的语言建模：

$$\mathcal{L}(\theta) = - \sum_{t=1}^T \log p_{\theta}(w_t | w_1, \dots, w_{t-1})$$

微调方式与 BERT 类似：取序列最后一个 token 的隐状态 h_T ，接一个线性层 $y \sim Ah_T + b$ 进行分类。

5.2 GPT-1：开创性的预训练 Decoder

GPT (Generative Pre-Training, 2018, OpenAI) 是第一个大规模预训练的 Decoder-only 模型：

- 117M 参数，12 层，768 维隐层
- 约 40,000 词的 BPE 词表
- 在 BookCorpus 上训练

GPT 在多项 NLP 基准上取得了强劲表现。BERT 紧随其后发布，凭借双向上下文在分类任务上略胜一筹，但 GPT 的生成能力是 BERT 无法匹敌的。

5.3 GPT-2：惊人的生成能力

GPT-2 (2019) 将模型规模提升至 **1.5B 参数**，训练数据增至约 90 亿词。

GPT-2 的生成质量在当时令人震惊，它能够产生**多段落、主题连贯**的文本，且 1.5B 的规模仍然可以在小型 GPU 上进行微调。

⁰Slides 第 51 页。来自 Radford et al., 2019。

5.4 GPT-3: 涌现的上下文学习能力

GPT-3 (2020) 实现了参数规模的巨大飞跃:

模型	参数量	训练数据	年份
GPT-1	117M	BookCorpus	2018
GPT-2	1.5B	WebText (9B 词)	2019
GPT-3	175B	300B 词	2020

表 1: GPT 系列模型的规模演进

GPT-3 最令人惊讶的不是它的微调性能, 而是它展现出了一种全新的能力——**上下文学习** (In-Context Learning, ICL)。

上下文学习 (In-Context Learning)

无需任何梯度更新或微调, 仅通过在 prompt 中给出几个**示例** (demonstrations), 模型就能“学会”执行新任务。这完全是通过**模式匹配**实现的: 模型识别出 prompt 中的输入-输出模式, 并将其应用于新的输入。

GPT-3 的上下文学习能力涵盖:

- 翻译 (给出几对翻译示例)
- 算术 (给出几个加法示例)
- 纠错 (给出几个错别字修正示例)
- 以及更多...

Chris Manning 强调这种能力是 “qualitatively new behavior”——从小模型中完全无法预见。GPT-3 只是被训练来预测下一个词, 但它**涌现**出了执行复杂任务的能力。这既令人兴奋, 也令人不安, 因为我们无法准确预测什么能力会涌现、什么时候会涌现。

5.5 Scaling Laws 与计算效率

Chinchilla Scaling Laws

DeepMind 的 Chinchilla 研究 (Hoffmann et al., 2022) 发现 GPT-3 的参数量与训练数据量之间**严重不平衡**——GPT-3 “comically oversized”。Chinchilla 只有 70B 参数 (不到 GPT-3 的一半), 但在 1.4T token 上训练 (GPT-3 只用了 300B), 结果性能更优。

核心结论: 给定固定计算预算, 参数量 N 和训练 token 数 D 应该**等比例**缩放: $N \propto D$ 。大约每个参数对应 20 个训练 token。

5.6 Chain-of-Thought 提示

⁰Slides 第 53 页。来自 Brown et al., 2020。

⁰Slides 第 56 页。来自 Kaplan et al., 2020。

⁰Slides 第 59 页。来自 Wei et al., 2022。

Chain-of-Thought Prompting 的原理

标准 prompting: 问题 → 答案

Chain-of-Thought: 问题 → **推理步骤** → 答案

关键洞察: 模型在生成每个 token 时能**条件于之前生成的所有 token**。通过先生成中间推理步骤, 模型:

- 获得了额外的“思考空间”(类似草稿纸)
- 将复杂问题分解为更简单的子问题
- 在每个子步骤中能利用之前步骤的结论作为条件

这显著提升了数学推理、逻辑推理等需要多步思考的任务的准确率。

5.7 为什么 Decoder-only 模型占据主导地位?

尽管 Encoder 和 Encoder-Decoder 各有优势, 当今最大、最强的预训练模型几乎都是 Decoder-only 架构。Chris Manning 指出原因可能包括:

1. **简洁性**: Decoder-only 架构更简单, 所有参数共享在一个网络中
2. **参数效率**: Encoder-Decoder 需要将参数分配给两个子网络, 而 Decoder-only 的所有参数都用于同一个任务
3. **统一性**: 所有任务(理解 + 生成)都可以建模为“给定 prompt, 生成 completion”
4. **涌现能力**: 上下文学习等涌现能力在 Decoder-only 模型中表现最为突出

5.8 本章小结

Decoder-only 预训练以 GPT 系列为代表, 通过简单的语言建模目标学习强大的语言能力。从 GPT-1 (117M) 到 GPT-3 (175B), 模型规模的增长带来了质变式的涌现能力, 尤其是上下文学习。Chinchilla 揭示了“更大不一定更好”的计算效率规律。Chain-of-Thought prompting 则展示了引导模型进行多步推理的有效策略。Decoder-only 架构已成为当今大语言模型的主流选择。

6 预训练数据与伦理考量

6.1 预训练数据的来源

主要的预训练数据来源包括:

- **CommonCrawl**: 互联网爬取数据(量最大, 质量参差不齐)
- **Wikipedia**: 高质量百科全书文本
- **BookCorpus / Books**: 书籍语料
- **WebText / OpenWebText**: 从 Reddit 高赞链接中收集的网页
- **学术论文** (ArXiv、PubMed)
- **代码** (GitHub、StackExchange)

6.2 预训练学到了什么——以及它的阴暗面

预训练的伦理风险

Chris Manning 特别强调：模型在学习有用知识的同时，也会 “learn and exacerbate racism and sexism, all manner of biases”。这些偏见来自训练数据中的不平衡和有害内容，而模型会忠实地复制甚至放大这些偏见。

这引出了关于预训练的重要伦理问题：

- 训练数据的筛选和清洗至关重要
- 模型的输出不应被盲目信任
- 部署前需要仔细评估模型在不同群体上的表现差异
- 大模型的能力边界和失败模式仍不完全清楚

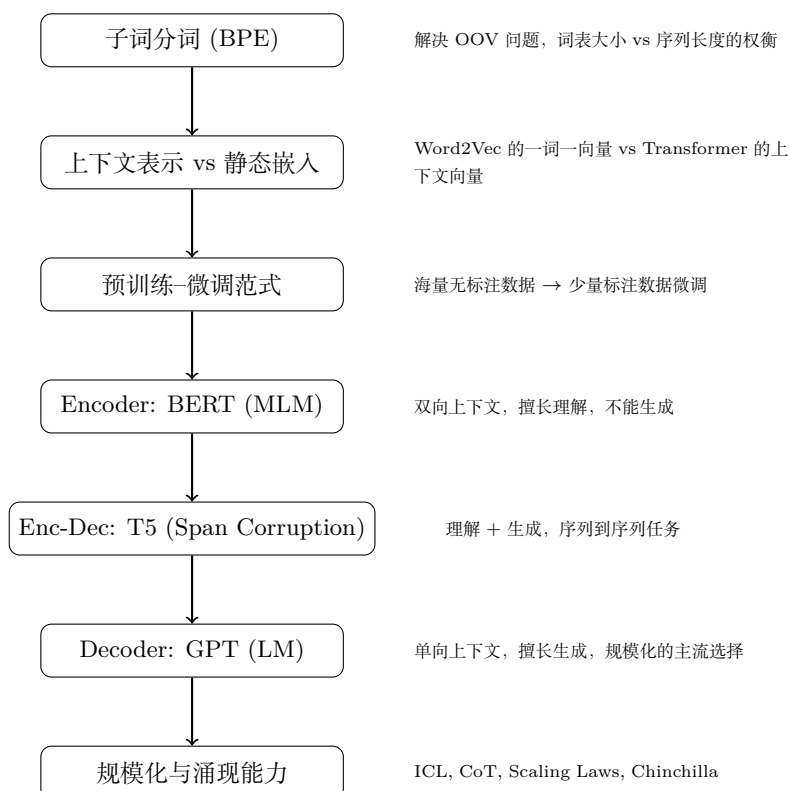
6.3 本章小结

预训练数据的规模和多样性是模型能力的基础，但数据质量和伦理问题同样不可忽视。随着模型越来越大、能力越来越强，理解它们的工作原理和失败模式变得愈发重要——这不仅是为了用好它们，更是为了负责任地部署它们。

7 总结与延伸

7.1 核心知识图谱

本节课建立了从词表示到大语言模型的完整认知链：



7.2 关键 Takeaways

本课七条核心原则

1. 子词分词解决了开放词表问题: BPE 等算法使模型能够处理任意输入, 消除了 UNK 标记
2. 上下文表示优于静态嵌入: 同一个词在不同上下文中应有不同的向量表示
3. 预训练-微调是当今 NLP 的基础范式: 先在海量无标注文本上学习通用表示, 再用少量标注数据适配具体任务
4. 三种架构各有所长: Encoder (理解)、Encoder-Decoder (理解 + 生成)、Decoder (生成), 但 Decoder-only 正成为主流
5. 规模带来涌现能力: 上下文学习、Chain-of-Thought 推理等能力在大规模模型中“意外”出现
6. 计算效率很重要: Chinchilla 表明参数量与数据量应等比例缩放, 盲目增大模型不一定最优
7. 能力与风险并存: 预训练模型学到的知识包括有害偏见, 部署时必须谨慎评估

7.3 拓展阅读

- Devlin et al., **BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding** (2018): <https://arxiv.org/abs/1810.04805>
- Liu et al., **RoBERTa: A Robustly Optimized BERT Pretraining Approach** (2019): <https://arxiv.org/abs/1907.11692>
- Joshi et al., **SpanBERT: Improving Pre-training by Representing and Predicting Spans** (2020): <https://arxiv.org/abs/1907.10529>
- Raffel et al., **Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer** (T5, 2020): <https://arxiv.org/abs/1910.10683>
- Radford et al., **Improving Language Understanding by Generative Pre-Training** (GPT, 2018)
- Brown et al., **Language Models are Few-Shot Learners** (GPT-3, 2020): <https://arxiv.org/abs/2005.14165>
- Hoffmann et al., **Training Compute-Optimal Large Language Models** (Chinchilla, 2022): <https://arxiv.org/abs/2203.15556>
- Wei et al., **Chain-of-Thought Prompting Elicits Reasoning in Large Language Models** (2022): <https://arxiv.org/abs/2201.11903>
- Hu et al., **LoRA: Low-Rank Adaptation of Large Language Models** (2021): <https://arxiv.org/abs/2106.09685>

- Gururangan et al., **Don't Stop Pretraining** (2020): <https://arxiv.org/abs/2004.10964>